Volume 4 – Tool SDK Developer Guide

Version: 1.0

Purpose

This document defines the Tool SDK used by developers to build reusable, secure, testable, and observable tools for the Enterprise Tiny Agent Framework.

This guide covers:

- Tool architecture
- Tool lifecycle
- Tool contracts
- Tool registration
- Security
- Versioning
- Testing
- Observability
- Tool composition

Audience:

- Java Developers
- Platform Engineers
- AI Engineers

Philosophy

In this architecture:

```
Agent = Decision Maker

Tool = Work Executor
```

Agents should remain thin.

Tools should contain business logic.

Golden Rule

If code accesses:

```
Database
Filesystem
REST API
Search Engine
Message Queue
Metadata Repository
```

It belongs in a Tool.

Not in an Agent.

Tool Design Principles

Principle 1: Single Responsibility

Each tool should perform one operation.

Bad:

```
MetadataTool
```

Responsibilities:

```
Search Dataset
Get Dataset
Get Columns
Get Lineage
Delete Dataset
```

Good:

```
SearchDatasetTool

GetDatasetTool
```

```
GetColumnsTool

GetLineageTool
```

Benefits:

- Easier testing
- Easier security
- Better model selection
- Better observability

Principle 2: Deterministic Behavior

Given identical inputs:

```
Input A
```

Tool must produce:

```
Output B
```

Every time.

Avoid:

```
Random behavior
Implicit state
Hidden dependencies
```

Principle 3: Structured Inputs

Never accept free-form text.

Bad:

```
String query
```

Good:

```
SearchDatasetRequest
```

Tool Architecture

```
Tool Request
      ↓
Validation
      ↓
Execution
      ↓
Transformation
      ↓
Tool Result
```

Core Interface

Tool

```
public interface Tool {

    String name();

    ToolResult execute(
            ToolRequest request);

}
```

Generic Request

```
public interface ToolRequest {
}
```

Generic Result

```
public interface ToolResult {
}
```

Strongly Typed Tool Pattern

Preferred approach:

```
public interface TypedTool<
        REQUEST extends ToolRequest,
        RESULT extends ToolResult> {

    RESULT execute(
            REQUEST request);

}
```

Example:

```
public interface SearchDatasetTool
    extends TypedTool<
        SearchDatasetRequest,
        SearchDatasetResult> {
}
```

Tool Lifecycle

Every Tool follows:

```
Receive Request
      ↓
Validate
      ↓
Authorize
      ↓
Execute
      ↓
Transform
      ↓
Audit
      ↓
Return
```

Abstract Tool

Provide a framework base class.

```
public abstract class AbstractTool
        implements Tool {

    @Override
    public ToolResult execute(
            ToolRequest request) {

        validate(request);

        authorize(request);

        ToolResult result =
                doExecute(request);

        audit(request, result);

        return result;
    }

    protected abstract ToolResult doExecute(
```

```
            ToolRequest request);

}
```

Validation

Never execute invalid requests.

Example:

```
protected void validate(
        ToolRequest request) {

    if(request == null) {
        throw new ValidationException();
    }
}
```

Authorization

Tools enforce permissions.

Example:

```
protected void authorize(
        ToolRequest request) {

    if(!securityService.canExecute(
            request)) {

        throw new AuthorizationException();
    }
}
```

Tool Categories

Metadata Tools

Purpose:

```
Dataset Discovery
Schema Lookup
Lineage Lookup
```

Examples:

```
SearchDatasetTool

GetDatasetTool

GetSchemaTool

GetLineageTool
```

SQL Tools

Purpose:

```
SQL Operations
```

Examples:

```
GenerateSqlTool

ValidateSqlTool

ExplainSqlTool
```

DuckDB Tools

Purpose:

```
Data Access
```

Examples:

```
ExecuteQueryTool

PreviewDatasetTool

DescribeTableTool
```

Filesystem Tools

Purpose:

```
Storage Access
```

Examples:

```
ListFilesTool

ReadFileTool

FileMetadataTool
```

Operations Tools

Purpose:

```
Job Management
```

Examples:

```
GetJobStatusTool

RestartJobTool
```

```
GetJobLogsTool
```

Research Tools

Purpose:

```
External Information Retrieval
```

Examples:

```
WebSearchTool

PageReaderTool

DocumentSearchTool
```

Tool Registry

All tools must be discoverable.

Registry Interface

```
public interface ToolRegistry {

    Tool getTool(
            String toolName);

}
```

Spring Implementation

```
@Component
public class DefaultToolRegistry
        implements ToolRegistry {
```

```
    private Map<String, Tool> tools;

}
```

Startup registration:

```
applicationContext
    .getBeansOfType(Tool.class);
```

Tool Definition

Every Tool should publish metadata.

```
public class ToolDefinition {

    private String id;

    private String name;

    private String description;

    private String version;

    private Set<String> tags;

}
```

Example:

```
ToolDefinition.builder()
        .id("search-dataset")
        .version("1.0")
        .build();
```

Tool Request Models

Every Tool owns request classes.

Example:

```
public record SearchDatasetRequest(

        String searchText,

        Integer limit

) implements ToolRequest {
}
```

Tool Result Models

Every Tool owns result classes.

Example:

```
public record SearchDatasetResult(

        List<Dataset> datasets

) implements ToolResult {
}
```

Error Handling

Never return exceptions directly.

Bad:

```
SQLException
```

Preferred:

```
ToolExecutionException
```

Structure:

```
public class ToolExecutionException
        extends RuntimeException {

    private String errorCode;

    private String message;

}
```

Retry Strategy

Only retry transient failures.

Examples:

```
Network Failure

Connection Timeout

Temporary Service Failure
```

Do not retry:

```
Validation Errors

Authorization Errors

Bad Input
```

Tool Composition

A Tool may call another Tool.

However:

```
Tool → Tool
```

is allowed.

```
Tool → Agent
```

is forbidden.

Example:

```
GenerateSqlTool
       ↓
MetadataLookupTool
```

Valid.

Caching

Recommended for:

```
Metadata Lookup

Schema Retrieval

Documentation Retrieval
```

Avoid caching:

```
Job Status

Execution Logs
```

Tool Observability

Every Tool execution must generate:

```
TOOL_STARTED

TOOL_COMPLETED

TOOL_FAILED
```

Metrics:

```
Execution Time

Success Rate

Failure Rate

Invocation Count
```

Tool Audit Model

```
public class ToolAuditEvent {

    private String eventId;

    private String toolName;

    private String requestId;

    private Instant timestamp;

    private Long durationMs;
```

```
}
```

Tool Security Levels

Recommended:

```
READ

WRITE

ADMIN
```

Examples:

READ

```
Search Dataset

Read File

Get Job Status
```

WRITE

```
Update Metadata

Execute Workflow
```

ADMIN

```
Delete Dataset

Restart Production Job
```

Dangerous Tools

Require approval.

Examples:

```
DeleteFileTool

DeleteDatasetTool

RestartProductionJobTool
```

Execution pattern:

```
Request
   ↓
Approval
   ↓
Execution
```

Tool Configuration

Externalize configuration.

Example:

```
tool:

  search-dataset:

    timeout: 10s

    cache-enabled: true

    cache-ttl: 30m
```

Avoid hardcoded values.

Tool Testing

Unit Testing

Mock:

```
Database

Filesystem

Remote APIs
```

Test:

```
Validation

Authorization

Execution
```

Integration Testing

Use:

```
TestContainers

Embedded Databases

Local Services
```

Contract Testing

Verify:

```
Input Schema

Output Schema
```

```
Error Schema
```

remain stable.

Versioning Strategy

Every Tool should be versioned independently.

Example:

```
search-dataset-v1

search-dataset-v2
```

Agents should explicitly declare supported versions.

MCP Readiness

Future MCP support becomes easy if every Tool already exposes:

```
ToolDefinition

Request Schema

Response Schema
```

The MCP adapter becomes:

```
Tool
    ↓
MCP Tool
```

without rewriting business logic.

Recommended Tool Package Structure

```
tool-runtime
│
├── tool-api
│
├── tool-core
│
├── metadata-tools
│
├── sql-tools
│
├── filesystem-tools
│
├── operations-tools
│
├── research-tools
│
└── tool-starter
```

Example SearchDatasetTool

Capability:

```
Search metadata catalog
```

Input:

```
SearchDatasetRequest
```

Output:

```
SearchDatasetResult
```

Security:

```
READ
```

Caching:

```
Enabled
```

Retry:

```
Disabled
```

Agent Usage:

```
Metadata Agent

SQL Agent

Documentation Agent
```

Tool Development Checklist

Before production:

- Single responsibility verified
- Request schema defined
- Result schema defined
- Validation implemented
- Authorization implemented
- Metrics enabled
- Audit enabled
- Unit tests complete
- Integration tests complete
- Configuration externalized
- Version assigned

Final Rule

Tools are the foundation of the platform.

A well-designed Tool should:

- Perform one operation

- Be independently testable
- Be independently deployable
- Be independently observable
- Be reusable by many agents

If multiple agents need the same capability, build a Tool.

If only one agent needs the capability, build a Tool anyway.

Business logic belongs in Tools. Agents should primarily orchestrate, plan, and decide.